

Дисциплина: Программирование корпоративных систем

Практическое занятие №3

Паттерны проектирования. Системы сборки

Техническая документация проекта

Тайм-трекер

Консольное приложение для учета рабочего времени

Выполнил: Лобин Александр Ильич

Группа: ЭФБО-06-24

Москва, 2026

1. Описание проекта

Тайм-трекер - консольное приложение на C++ для учета рабочего времени по задачам. Пользователь может создавать задачи, запускать и останавливать таймер, просматривать историю рабочих сессий и получать отчет о суммарном времени.

1.1 Основные функции

- создание задач с названием и описанием;
- просмотр списка активных и архивных задач;
- запуск рабочей сессии для выбранной задачи;
- остановка активной сессии и расчет длительности;
- сохранение данных в локальные файлы;
- формирование отчета по затраченному времени.

1.2 Основные классы

Класс	Назначение
Task	Доменная модель задачи: идентификатор, название, описание, статус.
TimeSession	Доменная модель рабочей сессии: задача, время начала, время завершения, активность.
TaskManager	Создание, поиск, просмотр и архивирование задач.
TimeTracker	Запуск и остановка таймера, контроль единственной активной сессии.
Storage	Загрузка и сохранение задач и сессий в локальные CSV-файлы.
ReportService	Агрегация завершенных сессий и расчет суммарного времени по задачам.
ConsoleUI	Пользовательский интерфейс командной строки.

2. Паттерны проектирования

Для проекта применимы несколько изученных паттернов. Они помогают разделить ответственность между модулями, упростить замену реализации хранения данных и сделать пользовательские команды более понятными для расширения.

2.1 Facade

Паттерн **Facade** применим к классу `ConsoleUI`. Пользователь работает с простым меню, а внутри интерфейс обращается к нескольким подсистемам: `TaskManager`, `TimeTracker`, `Storage` и `ReportService`.

Например, команда остановки таймера вызывает `TimeTracker::stop()`, затем пользователь может сохранить данные через `Storage::saveSessions()`. Внешнему пользователю не нужно знать, какие объекты участвуют в этом сценарии: он выбирает пункт меню, а фасад координирует работу подсистем.

2.2 Repository

Паттерн **Repository** применим к классу `Storage`. Он отделяет доменную логику от деталей хранения данных. Классы `TaskManager` и `TimeTracker` работают с объектами `Task` и `TimeSession`, а не с файлами напрямую.

В текущей версии `Storage` сохраняет данные в файлы `tasks.csv` и `sessions.csv`. В дальнейшем реализацию можно заменить на SQLite или JSON, не меняя основную бизнес-логику приложения.

2.3 Singleton

Паттерн **Singleton** может быть применен к сервису конфигурации приложения, например к будущему классу `AppConfig`. Такой класс будет хранить путь к каталогу данных, формат вывода времени и другие глобальные настройки.

В текущей реализации Singleton намеренно не добавлен в код, потому что проект небольшой и достаточно передавать объект `Storage` через конструктор. Однако при появлении единого конфигурационного объекта Singleton будет полезен для централизованного доступа к неизменяемым настройкам.

2.4 Strategy

Паттерн **Strategy** применим для формирования отчетов. Сейчас `ReportService` строит общий отчет по всем завершенным сессиям. В дальнейшем можно выделить интерфейс `ReportStrategy` и реализации `DailyReportStrategy`, `TaskReportStrategy`, `PeriodReportStrategy`.

Например, пользователь выбирает тип отчета, а `ReportService` получает конкретную стратегию и вызывает общий метод генерации. Это позволит добавлять новые виды отчетов без переписывания существующего кода.

2.5 Command

Паттерн **Command** применим к обработке пунктов меню. Сейчас команды реализованы методами `ConsoleUI::addTask()`, `ConsoleUI::startTimer()`, `ConsoleUI::stopTimer()` и другими. При росте интерфейса каждое действие можно оформить отдельным объектом-командой.

Например, класс `StartTimerCommand` будет хранить ссылку на `TimeTracker` и идентификатор задачи, а метод `execute()` будет запускать таймер. Такой подход упростит добавление истории команд, отмены операций и тестирование пользовательских сценариев.

2.6 Factory Method

Паттерн **Factory Method** применим при создании объектов рабочих сессий и отчетов. Например, будущий класс `SessionFactory` может создавать корректно заполненный объект `TimeSession` с уникальным идентификатором и текущим временем начала.

Это полезно, потому что правила создания сессии не будут размазаны по проекту. Если позже понадобится добавить тип сессии, теги или источник запуска, изменение останется внутри фабрики.

3. Организация файловой структуры

Проект организован по модульному принципу. Заголовочные файлы находятся в каталоге `include/`, реализации - в каталоге `src/`.

```
KR3/  
  CMakeLists.txt  
  README.md  
  include/  
    Task.h  
    TimeSession.h  
    TaskManager.h  
    TimeTracker.h  
    Storage.h  
    ReportService.h  
    ConsoleUI.h  
  src/  
    main.cpp  
    Task.cpp  
    TimeSession.cpp  
    TaskManager.cpp  
    TimeTracker.cpp  
    Storage.cpp  
    ReportService.cpp  
    ConsoleUI.cpp
```

4. Система сборки CMake

Для проекта подготовлен корневой файл `CMakeLists.txt`. Он задает имя проекта, минимальную версию CMake, стандарт C++17, список исходных файлов и каталог заголовков.

```
cmake_minimum_required(VERSION 3.10)
project(TimeTracker LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 17)
add_executable(time_tracker)
target_sources(time_tracker PRIVATE ...)
target_include_directories(time_tracker PRIVATE include)
```

4.1 Команды сборки

```
cmake -S . -B build
cmake --build build
cd build
./time_tracker
```

4.2 Сторонние библиотеки

Сторонние библиотеки не используются. Для хранения данных, работы со временем, файловой системой и коллекциями применяется стандартная библиотека C++17.

5. Вывод

В проект добавлена глава с анализом применимых паттернов проектирования. Также подготовлена модульная C++ структура, настроен `CMakeLists.txt`, добавлен `README.md` с инструкцией по сборке и запуску. Проект соответствует требованиям практического занятия: код разделен на логические модули, сборка описана через CMake, а техническая документация оформлена как HTML и PDF.